

Section 4: Volumes and Data Persistence

Section Goal

By the end of this section, you should be able to say:

"I understand how to persist data in containers and why volumes are essential for databases."

This section solves the data loss problem from Section 3.

The Problem We Need to Solve

In Section 3, we ran PostgreSQL and discovered:

- When you remove a container, all data is lost
- Databases are stateful—they need to keep data
- Containers are ephemeral by design

How do we run a database that keeps its data?

Why Containers Lose Data

Important reality:

- Containers have their own filesystem
- When a container is removed, its filesystem is deleted
- This is by design—containers are meant to be disposable

But applications often need to:

- Store database files
 - Persist logs
 - Save uploaded files
 - Retain configuration
-

Volumes (The Solution)

Volumes allow us to:

- Store data **outside** the container
- Reuse data across container restarts and removals
- Separate application (ephemeral) from data (persistent)

Think of volumes as:

"A safe place for data that survives the container."

Hands-On: PostgreSQL With a Volume

Lab Objective

Run PostgreSQL with a volume so that data persists even when the container is stopped and removed.

Step 1: Run PostgreSQL Without a Volume (Recap)

Run:

```
docker run -p 5432:5432 -e POSTGRES_PASSWORD=mypassword postgres
```

SHELL

In another terminal, connect and create a table:

```
psql -h localhost -U postgres -W
```

SHELL

Alternative way to connect to the database

```
docker exec -it <container_id_or_name> psql -U postgres
```

SHELL

```
CREATE TABLE test (id SERIAL PRIMARY KEY, name VARCHAR(100));

INSERT INTO test (name) VALUES ('Ivo Costa');

SELECT * FROM test;
```

Stop and remove the container. The data is gone.

👣 Step 2: Run PostgreSQL With a Volume

Stop the previous container, then run:

```
docker run -p 5432:5432 -e POSTGRES_PASSWORD=mypassword -v
pgdata:/var/lib/postgresql postgres:18
```

SHELL

Explanation:

- `-v pgdata:/var/lib/postgresql/data` → Creates a volume named `pgdata` and mounts it to PostgreSQL's data directory
 - `pgdata` → Docker-managed volume (Docker creates and manages it)
 - `/var/lib/postgresql/data` → Where PostgreSQL stores its data inside the container
-

👣 Step 3: Create Data and Remove the Container

1. Connect with psql and create a table with data
2. Exit psql
3. Stop the container: `docker stop <container_id>`
4. Remove the container: `docker rm <container_id>`
5. Run a **new** container with the **same** volume:

```
docker run -p 5432:5432 -e POSTGRES_PASSWORD=mypassword -v  
pgdata:/var/lib/postgresql/data postgres
```

6. Connect again and run: `SELECT * FROM test;`

Observation: The data is still there.

Key Learning Moment

The volume `pgdata` exists independently of the container.

- Container 1 created data → removed
- Container 2 used same volume → data preserved

Volumes outlive containers.

Volume Types (Brief Overview)

Named volumes (what we used):

- `-v pgdata:/var/lib/postgresql/data`
- Docker manages the storage location
- Easy to use, portable
- Best for databases

Bind mounts (advanced, for later):

- `-v /home/user/data:/app/data`
- Maps a host directory into the container
- Useful for development (live code sync)

For now, named volumes are sufficient.

Inspecting Volumes

View existing volumes:

SHELL

```
docker volume ls
```

Inspect a volume:

SHELL

```
docker volume inspect pgdata
```

Remove a volume (only when no container is using it):

SHELL

```
docker volume rm pgdata
```

Important: Same Password, Same Volume

When reusing a PostgreSQL volume:

- Use the **same** `POSTGRES_PASSWORD` as when the volume was first created
- PostgreSQL stores credentials in the volume
- Different password = PostgreSQL may fail to start (data directory ownership)

Mini Exercise (Student Activity)

1. Run PostgreSQL with a volume
2. Create a database and add several records
3. Stop and remove the container
4. Start a new container with the same volume
5. Verify all data is still present
6. Run `docker volume ls` to see the volume exists

Goal:

- Reinforce that volumes persist data
 - Build confidence with the volume lifecycle
-

⚠️ Common Beginner Mistakes

- Forgetting `-v` when running a database → Data lost on container removal
 - Using different volume names → Creates separate volumes, no data reuse
 - Removing the volume with `docker volume rm` → All data permanently deleted
-

💡 Section Takeaway

You now understand:

- Why containers lose data
- What volumes are and why they matter
- How to run PostgreSQL with a volume
- That volumes persist beyond container lifecycle
- Basic volume inspection commands

Your databases can now survive container restarts and replacements.

📌 Conclusion

"We can run databases with persistent data.
Now let's package our own application into a Docker image."